Discussion of Exercise Sheet 3

Exercise 3.1 *Asymptotic growth.*

(a) Prove or disprove the following statements. Justify your answer.

(1) For all $a, b > 1$, suppose $n \in \mathbb{N}$ and $n > \max\{a, b\}$, then $\log_a(n) = \Theta(\log_b(n))$.

Let $a, b > 1$ be arbitrary

$$\log_a(n) = \frac{\log_b(n)}{\log_b(a)} = \frac{1}{\log_b(a)} \cdot \log_b(n) = \Theta(\log_b(n))$$

Basiswechsel — konstanter Faktor

Fazit: Die Basis spielt asymptotisch keine Rolle. Alle Logarithmen wachsen asymptotisch gleich schnell.

(2) For all $a, b \geq 1$, suppose $n \in \mathbb{N}$, then $a^n = \Theta(b^n)$ (Master Solution ist bisschen komisch)

Gegenbeispiel: $a = 1$, $b = 2$

$$\lim_{n\to\infty} \frac{a^n}{b^n} = \lim_{n\to\infty} \frac{1^n}{2^n} = \lim_{n\to\infty} \frac{1}{2^n} = 0$$

$1^n = 1$

Theorem 1 $\Rightarrow$ $a^n \neq \Theta(b^n)$

(b) (1) Prove that $\lim_{n\to\infty} \frac{n}{\log(n)} = \infty$.

Hint: *Use L'Hôpital's rule.*

$$\lim_{n\to\infty} \frac{n}{\log(n)} = \lim_{n\to\infty} \frac{n}{\frac{\ln(n)}{\ln(2)}} \overset{\text{L'Hôp}}{=} \lim_{n\to\infty} \frac{1}{\frac{\frac{1}{n}}{\ln(2)}} = \lim_{n\to\infty} n \cdot \ln(2) = \infty$$

Basiswechsel — Hilfe: $\frac{\frac{A}{B}}{\frac{C}{D}} = \frac{A}{B} \cdot \frac{D}{C}$

(2) Prove that $\lim_{n\to\infty} n(e^{1/n} - 1) = 1$.

You may use the following variant of L'Hôpital's rule:

Theorem 2 (L'Hôpital's rule (going to 0)). *Assume that functions $f : \mathbb{R}^+ \to \mathbb{R}$ and $g : \mathbb{R}^+ \to \mathbb{R}$ are differentiable, $\lim_{x\to\infty} f(x) = \lim_{x\to\infty} g(x) = 0$ and for all $x \in \mathbb{R}^+$, $g'(x) \neq 0$. If $\lim_{x\to\infty} \frac{f'(x)}{g'(x)} = C \in \mathbb{R}$ or $\lim_{x\to\infty} \frac{f'(x)}{g'(x)} = \infty$, then*

$$\lim_{x\to\infty} \frac{f(x)}{g(x)} = \lim_{x\to\infty} \frac{f'(x)}{g'(x)}.$$

$\lim_{n\to\infty} n \cdot (e^{\frac{1}{n}} - 1) = \lim_{n\to\infty} \frac{e^{\frac{1}{n}} - 1}{\frac{1}{n}}$ Umformung weil wir die Form "$\frac{0}{0}$" für L'Hôpital brauchen

oben: $\frac{1}{n} \to 0$, deshalb $e^{\frac{1}{n}} \to e^0 = 1$, deshalb $e^{\frac{1}{n}} - 1 \to 0$ ✓

unten: $\frac{1}{n} \to 0$ ✓

Ableitung: $(\frac{1}{n})' = (n^{-1})' = -n^{-2} = -\frac{1}{n^2}$

$(e^{\frac{1}{n}})' = e^{\frac{1}{n}}$ $(\frac{1}{n})' = -e^{\frac{1}{n}} \cdot \frac{1}{n^2}$

Kettenregel

L'Hôp. $= \lim_{n\to\infty} \frac{-e^{\frac{1}{n}} \cdot \frac{1}{n^2}}{-\frac{1}{n^2}}$

$= \lim_{n\to\infty} e^{\underbrace{\frac{1}{n}}_{\to 0}}$

$= 1$

weil $\lim_{n\to\infty} e^{f(n)} = e^{\lim_{n\to\infty} f(n)}$

Das "Limes hineinziehen" ist nicht immer erlaubt,

Beispiel: $2 = \lim_{n\to\infty} 2 = \lim_{n\to\infty} 2^{n\frac{1}{n}} = \lim_{n\to\infty} (2^n)^{\underbrace{\frac{1}{n}}_{\to 0}} = \lim_{n\to\infty} (2^n)^0 = 1$

Regel für A&D: erlaubt wenn die Basis konstant ist

Anderer Lösungsweg mit Substitution $y = \frac{1}{n}$

$\lim_{n\to\infty} n \cdot (e^{\frac{1}{n}} - 1) = \lim_{y\to 0} \frac{1}{y} \cdot (e^y - 1) = \lim_{y\to 0} \frac{e^y - 1}{y} \overset{\text{L'Hôp}}{=} \lim_{y\to 0} \frac{e^y}{1} = 1$

Substitution $y = \frac{1}{n} \Leftrightarrow n = \frac{1}{y}$

wenn $n \to \infty$, dann $y = \frac{1}{n} \to 0$

weil $y \to 0$ folgt $e^y \to e^0 = 1$

(Ex 3 skipped)

Exercise 3.2 *Substring counting* **(2 bonus points).** → peer grading

Given an n-bit bitstring $S[0..n-1]$ (i.e. $S[i] \in \{0,1\}$ for $i = 0, 1, ..., n-1$), and an integer $k \geq 0$, we would like to count the number of nonempty contiguous substrings of S with exactly k ones. Assume $n \geq 2$.

For example, when $S =$ "0110" and $k = 2$, there are 4 such substrings: "011", "11", "110", and "0110".

(a) Design a "naive" algorithm that solves this problem with a runtime of $O(n^3)$. Describe the algorithm using pseudocode. Justify the runtime (you don't need to provide a formal proof, but you should state your reasoning).

Just iterate over all possible substrings from i to j and count the ones

Algorithm 1 Naive substring counting

```
c ← 0                                          ▷ Initialize counter of substrings with k ones
for i ← 0, ..., n − 1 do                       ▷ Enumerate all nonempty substrings S[i..j]
    for j ← i, ..., n − 1 do
        x ← 0                                  ▷ Initialize counter of ones
        for ℓ ← i, ..., j do                   ▷ Count ones in substring
            if S[ℓ] = 1 then
                x ← x + 1
        if x = k then                          ▷ If there are k ones in substring, increment c
            c ← c + 1
return c                                       ▷ Return number of substrings with k ones
```

Runtime: The nested for-loops have three levels and each level has at most n iterations, leading to $O(n^3)$ iterations in total. Each iteration runs in $O(1)$ time. Thus the total running time is $O(n^3)$.

(b) We say that a bitstring S' is a *(non-empty) prefix* of a bitstring S if S' is of the form $S[0..i]$ where $0 \leq i <$ length(S). For example, the prefixes of $S =$ "0110" are "0", "01", "011" and "0110".

Given an n-bit bitstring S, we would like to compute a table T indexed by $0..n$ such that for all i, $T[i]$ contains the number of prefixes of S with exactly i ones.

For example, for $S =$ "0110", the desired table is $T = [1, 1, 2, 0, 0]$, since, of the 4 prefixes of S, 1 prefix contains zero "1", 1 prefix contains one "1", 2 prefixes contain two "1", and 0 prefixes contain three "1" or four "1".

Design an algorithm PREFIXTABLE that computes T from S in time $O(n)$, assuming S has size n. Describe the algorithm using pseudocode. Justify the runtime (you don't need to provide a formal proof, but you should state your reasoning).

Algorithm 2

```
function PREFIXTABLE(S)
    T[0..n] ← an array of (n + 1) zeros          ▷ Initialize array
    s ← 0
    for i ← 0, ..., n − 1 do                     ▷ Enumerate all prefixes S[0..i]
        s ← s + S[i]                             ▷ s saves the number of "1" in S[0..i]   = number of 1s seen so far
        T[s] ← T[s] + 1                          ▷ S[0..i] is a prefix with s "1"
    return T
```

- we have exactly one new prefix for every i
- s stores the number of 1s of this prefix
- Therefore we add 1 to T[s], because we have seen one more prefix with s 1s

Remark: This algorithm can also be applied on a reversed bitstring to compute the same table for all suffixes of S. In the following, you can assume an algorithm SUFFIXTABLE that does exactly this.

(difficult)

(c) Consider an integer $m \in \{0, 1, \ldots, n-2\}$. Using PREFIXTABLE and SUFFIXTABLE, design an algorithm SPANNING(m, k, S) that returns the number of substrings $S[i..j]$ of S that have exactly k ones and such that $i \leq m < j$.

⇒ substring must include index m and m+1

For example, if $S =$ "0110", $k = 2$, and $m = 0$, there exist exactly two such strings: "011" and "0110". Hence, SPANNING$(m, k, S) = 2$.

Describe the algorithm using pseudocode. Mention and justify the runtime of your algorithm (you don't need to provide a formal proof, but you should state your reasoning).

Hint: *Each substring $S[i..j]$ with $i \leq m < j$ can be obtained by concatenating a string $S[i..m]$ that is a suffix of $S[0..m]$ and a string $S[m+1..j]$ that is a prefix of $S[m+1..n-1]$.*

S: [1, 0, . . , 0, 1, 1, 1, 0, 0, 1, . . , 1, 0]

i m m+1 j

← Suffixtable | Prefixtable →

p ones | k-p ones

total: p + k-p = k ones

Algorithm 3

function SPANNING(m, k, S)
  $T_1 \leftarrow$ SUFFIXTABLE$(S[0..m])$
  $T_2 \leftarrow$ PREFIXTABLE$(S[m+1..n-1])$
  return $\sum_{p=\max(0,k-(n-m-1))}^{\min(k,m+1)} (T_1[p] \cdot T_2[k-p])$

iterate over all possible p's

p ≤ k because we don't want more than k ones

p ≤ m+1 because the left side has length m+1

p ≥ 0 because we can't have a negative number of ones

p ≥ k - (n-m-1) because the right side has length (n-m-1)

Runtime: Computing both tables takes $O(n)$ time, and the sum has at most n terms, each taking $O(1)$ time. Thus the total runtime is $O(n)$.

Exercise 3.3 *Counting function calls in loops* **(2 bonus points).**

For each of the following code snippets, compute the number of calls to f as a function of $n \in \mathbb{N}$. Provide **both** the exact number of calls and a maximally simplified asymptotic bound in Θ notation.

(a)

Algorithm 1

```
i ← 0
while i ≤ n do
    f()
    f()
    i ← i + 1
j ← 0
while j ≤ 2n do
    f()
    j ← j + 1
```

Number of calls $= \sum_{i=0}^{n} 2 + \sum_{j=0}^{2n} 1$

$= (n+1) \cdot 2 + (2n+1) \cdot 1$

$= 2n + 2 + 2n + 1$

$= 4n + 3$

$= \Theta(4n)$

(b)

Algorithm 2

```
i ← 1
while i ≤ n do
    j ← 1
    while j ≤ i^3 do
        f()
        j ← j + 1
    i ← i + 1
```

Hint: *You may use* $\sum_{i=1}^{n} i^3 = \frac{n^2(n+1)^2}{4}$.

Number of calls $= \sum_{i=1}^{n} \sum_{j=1}^{i^3} 1$

$= \sum_{i=1}^{n} i^3$

$= \frac{n^2 (n^2+1)^2}{4}$

$= \Theta(n^4)$

→Ex 45

Suchen

LINEAR-SEARCH$(A[1..n], b)$

```
1 for i ← 1, 2, ..., n do
2     if A[i] = b then return i
3 return "nicht gefunden"
```

- Anwendbar für beliebige Arrays (auch unsortierte)
- Worst case Laufzeit: $\Theta(n)$

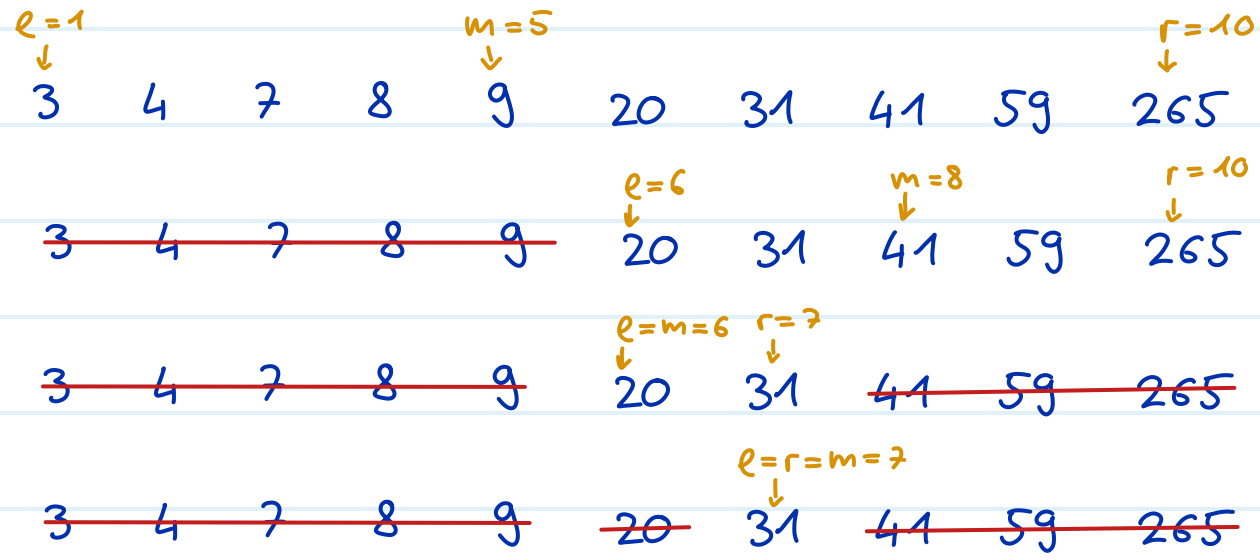
BINARY-SEARCH$(A[1..n], b)$

```
1 ℓ ← 1; r ← n                          ▷ Initialer Suchbereich
2 while ℓ ≤ r do
3     m ← ⌊(ℓ + r)/2⌋
4     if A[m] = b then return m          ▷ Element gefunden
5     else if A[m] > b then r ← m − 1    ▷ Suche links weiter
6     else ℓ ← m + 1                     ▷ Suche rechts weiter
7 return "Nicht vorhanden"
```

- Anwendbar nur für sortierte Arrays
- In jeder Iteration halbieren wir den Suchbereich
- Worst case Laufzeit: $\Theta(\log n)$

Beispiel:

BinarySearch(A,31):



Intuition für die Laufzeit:

- $\log(n)$ = " wie oft muss ich n halbieren bis ich bei 1 ankomme? " (für $n = 2^k$)
- Allgemein: $\log_a(n)$ = " wie oft muss ich n durch a teilen bis ich bei 1 ankomme? " (für $n = a^k$)

Bsp $\log(32) = 5$ weil $32 \xrightarrow{1} 16 \xrightarrow{2} 8 \xrightarrow{3} 4 \xrightarrow{4} 2 \xrightarrow{5} 1$

$\log(8) = 3$ weil $8 \xrightarrow{1} 4 \xrightarrow{2} 2 \xrightarrow{3} 1$

$\log(4) = 2$ weil $4 \xrightarrow{1} 2 \xrightarrow{2} 1$

$\log_3(27) = 3$ weil $27 \xrightarrow[1]{:3} 9 \xrightarrow[2]{:3} 3 \xrightarrow[3]{:3} 1$

Mehr Intuition für log Gesetze.

$\log(a \cdot b) = \log(a) + \log(b)$:

$\log(4 \cdot 8) = \log(4) + \log(8)$

$\log(4 \cdot 8)$: wie oft 4·8 = 32 halbieren bis 1

$\log(4)$: wie oft 4 halbieren bis 1

$\log(8)$: wie oft 8 halbieren bis 1

$\log((2\,2)\,(2\,2\,2)) = \log(2\,2) + \log(2\,2\,2)$

$5 = 2 + 3$

$\log_a(x) = \dfrac{\log_b(x)}{\log_b(a)}$.

$\log_4(64) = \dfrac{\log_2(64)}{\log_2(4)}$, $\quad 3 = \dfrac{6}{2}$

$\log_2(64) = 6$: $64 \xrightarrow{1} 32 \xrightarrow{2} 16 \xrightarrow{3} 8 \xrightarrow{4} 4 \xrightarrow{5} 2 \xrightarrow{6} 1$

$\log_4(64) = 3$: $64 \xrightarrow[1]{:4} 16 \xrightarrow[2]{:4} 4 \xrightarrow[3]{:4} 1$

$\log_2(4) = 2$: $4 \xrightarrow{:2} 2 \xrightarrow{:2} 1$, $4 \xrightarrow{:4}$

Korrektur-Faktor
weil der violette Pfeil genau zwei blauen entspricht.

$\log(a^b) = b \cdot \log(a)$:

$\log(4^3) = \log((2\,2)\ (2\cdot 2)\cdot(2\,2)) = \log(2\,2) + \log(2\,2) + \log(2\,2)$

$= 3 \cdot \log(2\,2)$

$\log(4^3)$: wie oft $4^3 = 64$ halbieren bis 1

$\log(2\,2)$: wie oft 4 halbieren bis 1

$6 = 3 \cdot 2$

$\log(4^3) = 6 \cdot$ 64 $\xrightarrow{1}$ 32 $\xrightarrow{2}$ 16 $\xrightarrow{3}$ 8 $\xrightarrow{4}$ 4 $\xrightarrow{5}$ 2 $\xrightarrow{6}$ 1

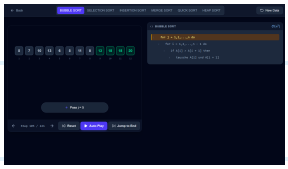
(1 | 2 | 3)

$\log(4) = 2$: $4 \xrightarrow{1} 2 \xrightarrow{2} 1$

Sortieren

BUBBLE-SORT($A[1..n]$)

```
1 for j ← 1, 2, ..., n do
2     for i ← 1, 2, ..., n − 1 do
3         if A[i] > A[i + 1] then
4             tausche A[i] und A[i + 1]
```



Vergleiche: $n \cdot (n-1) = \Theta(n^2)$

Vertauschungen: $O(n^2)$

Laufzeit: $\Theta(n^2)$

Inv(j) Nach j Iterationen der äusseren Schleife sind die j grössten Elemente am richtigen Ort. (d.h. an ihrem finalen Index)



Beweis-Schema für Sortier-Algorithmen: → Ex 4.4

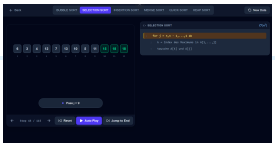
1.) Inv(j) = ... formulieren

2.) Base case: Inv(0) zeigen (oder Inv(1) je nach Algorithmus)

3.) I.H. Wir nehmen Inv(j) an für ein beliebiges $0 \leq j \leq n-1$ (Grenzen je nach Alg anpassen)

4) I.S. Unter dieser Annahme zeigen wir, dass nach einer weiteren Iteration (oder was auch immer zwischen Inv(j) und Inv(j+1) passiert) Inv(j+1) gilt. (Inv(j) => Inv(j+1))

5) Termination: Wir zeigen, dass falls Inv(n) gilt, dass dann das Array sortiert ist (d.h. $A[i] \leq A[i+1]$ für $1 \leq i \leq n-1$)



SELECTION-SORT($A[1..n]$)

```
1 for j ← n, n − 1, . . . , 1 do
2     k ← Index des Maximums in A[1, . . . , j]
3     tausche A[k] und A[j]
```

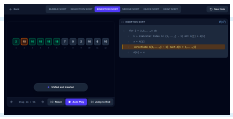
Vergleiche: $\sum_{j=1}^{n} j-1 = \sum_{j=0}^{n-1} j = \frac{n(n-1)}{2} = \Theta(n^2)$

(Index Shift; Gauss'sche Summe)

Vertauschungen: $O(n)$

Laufzeit: $\Theta(n^2)$

Inv (j): gleich wie Bubble-Sort: Nach j Iterationen der äusseren Schleife sind die j grössten Elemente am richtigen Ort.



INSERTION-SORT($A[1..n]$)

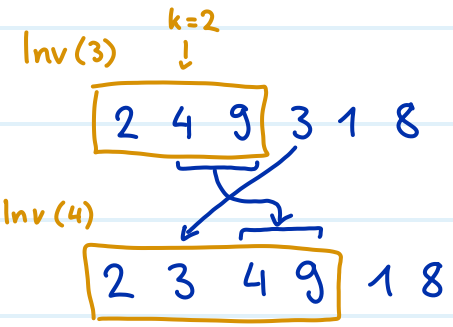
```
1 for j ← 2, 3, . . . , n do
2     k ← kleinster Index in {1, . . . , j − 1} mit A[j] ≤ A[k]   → mit BinarySearch
                                                  ▷ A[j] gehört an diese Stelle k
3     x ← A[j]                                    ▷ merke A[j]
4     verschiebe A[k, . . . , j − 1] nach A[k + 1, . . . , j]
5     A[k] ← x
```

Vergleiche: $O(n \log n)$ (für BinarySearch)

Vertauschungen: $O(n^2)$

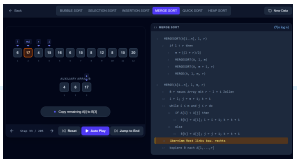
Laufzeit: $O(n^2)$

Bsp j=3



Inv (j): Nach j Iterationen der Schleife ist das Teilarray A[1..j] sortiert

(es muss aber nicht zwingend die j kleinsten Elemente enthalten)



```
MERGESORT(A[1..n], l, r)                          ▷ sortiert A[l, ..., r]
1  if l < r then
2      m ← ⌊(l + r)/2⌋
3      MERGESORT(A, l, m)                         ▷ sortiere linke Hälfte
4      MERGESORT(A, m + 1, r)                     ▷ sortiere rechte Hälfte
5      MERGE(A, l, m, r)                          ▷ verschmelze beide Hälften
```

divide and conquer

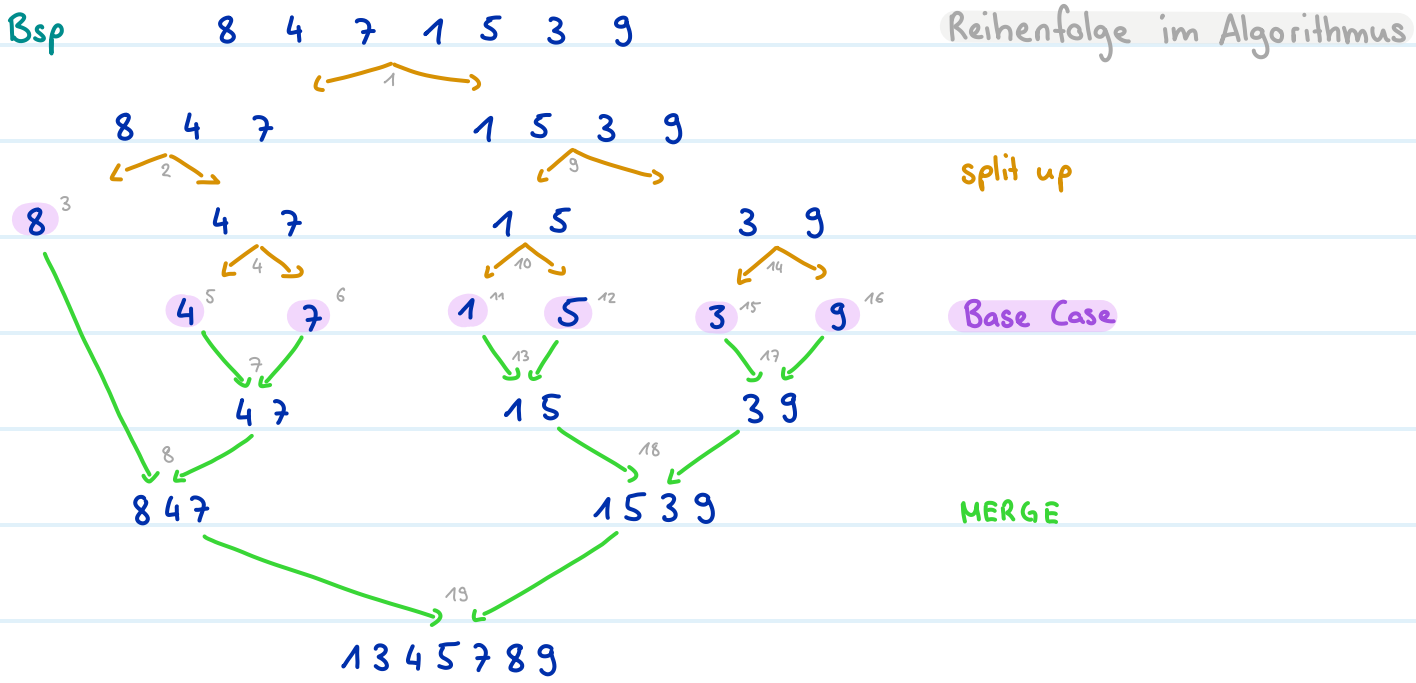
```
MERGE(A[1..n], l, m, r)
1  B ← neues Array mit r − l + 1 Zellen           ▷ so gross wie A[l, ..., r]
2  i ← l                                          ▷ erstes unbenutztes Element in linker Hälfte
3  j ← m + 1                                      ▷ erstes unbenutztes Element in rechter Hälfte
4  k ← 1                                          ▷ nächste Position in B
5  while i ≤ m and j ≤ r do                       ▷ beide Hälften noch nicht ausgeschöpft
6      if A[i] < A[j] then
7          B[k] ← A[i]
8          i ← i + 1
9          k ← k + 1
10     else
11         B[k] ← A[j]
12         j ← j + 1
13         k ← k + 1
14 übernimm Rest links bzw. rechts                ▷ wenn die andere Hälfte ausgeschöpft ist
15 kopiere B nach A[l, ..., r]
```

Laufzeit von MERGE $O(n)$

Totale Laufzeit: $T(n) \leq \underbrace{2 \cdot T\left(\frac{n}{2}\right)}_{\text{zwei Hälften rekursiv sortieren}} + \underbrace{d \cdot n}_{\text{für MERGE}}$ und $T(1) = c$

Teleskopieren $\Rightarrow$ $T(n) \leq O(\underbrace{n}_{\text{für MERGE}} \underbrace{\log n}_{\text{Tiefe des Rekursionsbaumes}})$

Zusätzlicher Speicher $O(n)$ für das Array B



Aufgaben:

g) *Sorting algorithms:*

i) Consider the sequence $6, 5, 4, 1, 2, 3$. How many swaps does `Bubble Sort` perform to sort this sequence? *Give the exact number of swaps required.*

ii) Consider the sequence $6, 5, 4, 1, 2, 3$. How many swaps does `Selection Sort` perform to sort this sequence? *Give the exact number of swaps required.*

iii) Let $n \in \mathbb{N}$ be an even number and consider the sequence with the following structure:

$$2, 1, 4, 3, 6, 5, \ldots, n, n-1.$$

How many swaps does `Insertion Sort` perform to sort this sequence? *Give the exact number, not just the asymptotics.*

True / False Questions:

- ☐ All four algorithms can sort any list of comparable elements correctly.
- ☐ Only Merge Sort can guarantee a worst-case time complexity better than $O(n^2)$.
- ☐ Bubble Sort compares distant elements in the array during each pass.
- ☐ Insertion Sort inserts each new element into the already sorted part at the final correct position.
- ☐ Selection Sort may perform fewer comparisons if the input list is already sorted.
- ☐ Merge Sort always splits the list into two equal halves.
- ☐ Bubble Sort and Insertion Sort both have an average runtime of $O(n \log n)$.
- ☐ Merge Sort's runtime does not depend on the initial order of the elements.
- ☐ Selection Sort performs fewer swaps than Bubble Sort in most cases.
- ☐ Insertion Sort requires extra arrays to perform its sorting.
- ☐ Merge Sort typically requires additional memory proportional to the size of the input.
- ☐ Selection Sort always performs exactly $n - 1$ comparisons.
- ☐ Bubble Sort can terminate early when no swaps occur during a pass.
- ☐ Merge Sort is an example of a "divide and conquer" algorithm.
- ☐ Bubble Sort is usually faster than Merge Sort on large, random datasets.
- ☐ Selection Sort adapts its runtime when the input is already sorted.
- ☐ Merge Sort can be implemented recursively or iteratively.
- ☐ Bubble Sort, Insertion Sort, and Selection Sort all sort in-place.
- ☐ Merge Sort sorts in-place without using additional arrays.
- ☐ All four algorithms determine order by comparing pairs of elements.
- ☐ Merge Sort divides the list until each sublist contains exactly one element.
- ☐ The worst-case runtime of Bubble Sort and Selection Sort is the same.
- ☐ Insertion Sort moves elements to make space for the current value being inserted.
- ☐ Merge Sort avoids element comparisons when merging sublists.
- ☐ Selection Sort and Bubble Sort perform the same number of swaps in total.
- ☐ Insertion Sort can outperform Merge Sort on very small arrays.
- ☐ Bubble Sort is primarily used in practical applications for large datasets.
- ☐ Any sorting algorithm is in $\Omega(n \log n)$.

- ☐ Selection Sort can be better than Bubble Sort.
- ☐ Merge Sort is always faster than Insertion Sort.
- ☐ Merge Sort has $O(n^2)$ comparisons.
- ☐ Insertion Sort runs in $O(n)$ time on sorted input.
- ☐ Bubble Sort's best case is faster than Selection Sort's best case.
- ☐ Merge Sort's runtime depends on how sorted the array already is.
- ☐ Selection Sort's number of comparisons depends on input order.
- ☐ Every comparison-based sort must perform $\Omega(n \log n)$ comparisons in every case.

Varia

Anmerkung zu Maximum Subarray Sum (Code Expert)

Variante 1: Teilproblem: $R[i]$:= grösste Summe eines Subarrays, das in Index i endet

Rekursion: $R[i+1] = \max\{a_{i+1}, R[i] + a_{i+1}\}$

Lösung: $\max_{1 \leq i \leq n} R_i$ oder 0 falls alle Arrayelemente negativ sind

Variante 2: Teilproblem: $R[i]$:= grösste Summe eines Subarrays, das in Index i endet, oder 0 falls sie negativ ist

Rekursion: $R[i+1] = \max\{0, R[i] + a_{i+1}\}$

Lösung: $\max_{1 \leq i \leq n} R_i$

Beispiel:

Array: [2, -5, 4, 1, -2, 7, -3, -14, 1, 2]

R mit Variante 1. [2, -3, 4, 5, 3, 10, 7, -7, 1, 3]

R mit Variante 2. [2, 0, 4, 5, 3, 10, 7, 0, 1, 3]

Formeln für Summen

Index Shift: $\sum_{i=a}^{b} f(i) = \sum_{i=a+c}^{b+c} f(i-c)$

Reverse $\sum_{i=a}^{b} f(i) = \sum_{i=a}^{b} f(a+b-i)$ und insbesondere $\sum_{i=0}^{n} f(i) = \sum_{i=0}^{n} f(n-i)$

Number of elements $\sum_{i=a}^{b} 1 = b-a+1$ (falls $b \geq a$)

Split up by range: $\sum_{i=a}^{b} f(i) = \sum_{i=a}^{m} f(i) + \sum_{i=m+1}^{b} f(i)$ (falls $a \leq m < b$)

Peeling off terms: $\sum_{i=a}^{b} f(i) = f(a) + \sum_{i=a+1}^{b} f(i) = \left(\sum_{i=a}^{b-1} f(i)\right) + f(b)$ (falls $a < b$)

Linearity: $\sum_{i=a}^{b} (f(i) \pm g(i)) = \sum_{i=a}^{b} f(i) + \sum_{i=a}^{b} g(i)$ and $\sum_{i=a}^{b} c \cdot f(i) = c \sum_{i=a}^{b} f(i)$

Sum of one element: $\sum_{i=a}^{a} f(i) = f(a)$

Index renaming: $\sum_{i=a}^{b} f(i) = \sum_{j=a}^{b} f(j)$

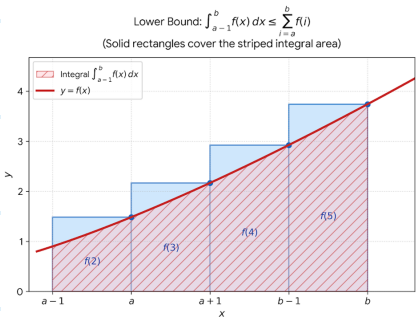
Swap order: $\sum_{i=a}^{b} \sum_{j=c}^{d} f(i,j) = \sum_{j=c}^{d} \sum_{i=a}^{b} f(i,j)$

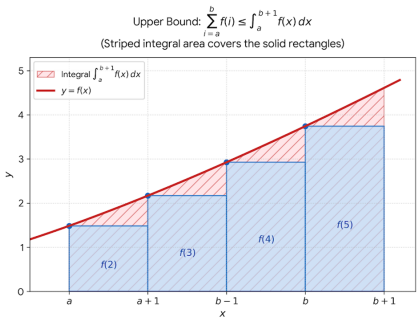
Triangular bounds: $\sum_{i=1}^{n} \sum_{j=1}^{i} f(i,j) = \sum_{j=1}^{n} \sum_{i=j}^{n} f(i,j)$

Upper bound: $\sum_{i=a}^{b} f(i) \leq M \cdot (b-a+1)$ falls $f(i) \leq M$ für $i = a, \ldots, b$

Lower bound $\sum_{i=1}^{n} f(i) \geq \sum_{i=\lceil \frac{n}{2} \rceil}^{n} f(i) \geq \underbrace{(n - \lceil \frac{n}{2} \rceil + 1)}_{\text{Anzahl Terme}} \cdot f\left(\lceil \frac{n}{2} \rceil\right) \geq \frac{n}{2} \cdot f\left(\frac{n}{2}\right)$ falls f monoton steigend ist

Integral trick: $\int_{a-1}^{b} f(x)\,dx \leq \sum_{i=a}^{b} f(i) \leq \int_{a}^{b+1} f(x)\,dx$ falls f monoton steigend ist





Aufgabe

Prove that $\sum_{i=1}^{n} \log(i^3) i^2 = \Theta(n^3 \log n)$

obere Schranke: $\sum_{i=1}^{n} \log(i^3) \cdot i^2 \leq$

untere Schranke: $\sum_{i=1}^{n} \log(i^3) \cdot i^2 \geq$

daraus folgt:

Neue Gruppen:

Group	Names	Emails
Group 1	Max Schaffner Charlyne Dippon	schaffnerma@student.ethz.ch cdippon@student.ethz.ch
Group 2	Gréta Frei Noa Kündig	grfrei@student.ethz.ch nkuendig@student.ethz.ch
Group 3	Lauren Thai Simon Spycher	lathai@student.ethz.ch sispycher@student.ethz.ch
Group 4	Besart Memisi Marc Leonard Overbeck	bmemisi@student.ethz.ch moverbeck@student.ethz.ch
Group 5	Paul Iurian Lucien Isler	piurian@student.ethz.ch islerlu@student.ethz.ch
Group 6	Raghav Tandon Sih-Cheng Chen	tandonr@student.ethz.ch chensih@student.ethz.ch
Group 7	Nicolas Fierz Floria Schwedt	nfierz@student.ethz.ch fschwedt@student.ethz.ch
Group 8	Durvish Battini Lucien Gees Noel Hobi	dbattini@student.ethz.ch lugees@student.ethz.ch hobino@student.ethz.ch
Group 9	Nico Reali Benjamin Stupf	nreali@student.ethz.ch bstupf@student.ethz.ch
Group 10	Elaine Banaag Aquino Silvan Wohlgensinger	ebanaag@student.ethz.ch swohlgensing@student.ethz.ch
Group 11	Nelio Gautschi Colin Jenny	ngautsch@student.ethz.ch cojenny@student.ethz.ch
Group 12	Matin Kamil Saih Zhentian Qi	kamilm@student.ethz.ch zhenqi@student.ethz.ch